```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m21.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m21.sh
m21.sh: line 9: char: command not found
m21.sh: line 11: syntax error near unexpected token `('
m21.sh: line 11: `void save_state()'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```
}

void display_directory()
{
    char current_directory[PATH_MAX];

    if (getcwd(current_directory,
               sizeof(current_directory)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("Current Directory: %s\n",
           current_directory);
}

int main()
{
    char input[MAX_INPUT];

    atexit(cleanup);

    printf("=============================================\n");
    printf("       DIRECTORY AND EXIT HANDLING\n");
    printf("=============================================\n");

    display_directory();

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0)
        {
            display_directory();
        }
```

```
    printf("       DIRECTORY AND EXIT HANDLING\n");
    printf("=========================================\n");

    display_directory();

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "pwd") == 0)
        {
            display_directory();
        }
        else if (strcmp(input, "exit") == 0)
        {
            printf("Exit request received.\n");
            break;
        }
        else if (strcmp(input, "save") == 0)
        {
            save_state();
            printf("State saved successfully.\n");
        }
        else if (strlen(input) == 0)
        {
            continue;
        }
        else
        {
            printf("Unknown command: %s\n",
                   input);
        }
    }

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m22.sh
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ bash m22.sh
m22.sh: line 10: syntax error near unexpected token `('
m22.sh: line 10: `int valid_variable_name(const char *name)'
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024

int valid_variable_name(const char *name)
{
    int i;

    if (name == NULL || name[0] == '\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) &&
        name[0] != '_')
    {
        return 0;
    }

    for (i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) &&
            name[i] != '_')
        {
            return 0;
        }
    }

    return 1;
}

void export_variable(char *input)
{
    char *command;
    char *assignment;
    char *name;
    char *value;

    command = strtok(input, " \t");

    if (command == NULL)
        return;

    if (strcmp(command, "export") != 0)
```

```
    char *value;

    command = strtok(input, " \t");

    if (command == NULL)
        return;

    if (strcmp(command, "export") != 0)
    {
        printf("Invalid command.\n");
        return;
    }

    assignment = strtok(NULL, " \t");

    if (assignment == NULL)
    {
        printf("Usage: export NAME=value\n");
        return;
    }

    name = strtok(assignment, "=");
    value = strtok(NULL, "");

    if (name == NULL || value == NULL)
    {
        printf("Invalid export syntax.\n");
        printf("Use: export NAME=value\n");
        return;
    }

    if (!valid_variable_name(name))
    {
        printf("Invalid variable name: %s\n",
               name);
        return;
    }

    if (setenv(name, value, 1) != 0)
    {
        perror("setenv");
        return;
    }

    printf("Exported successfully:\n");
    printf("%s=%s\n", name, getenv(name));
}
```

```c
void test_child_process(const char *variable)
{
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        char *value = getenv(variable);

        printf("\nChild process:\n");

        if (value != NULL)
        {
            printf("%s=%s\n",
                   variable,
                   value);
        }
        else
        {
            printf("%s is not available.\n",
                   variable);
        }

        exit(0);
    }

    waitpid(pid, &status, 0);
}

int main()
{
    char input[MAX_INPUT];

    printf("=============================================\n");
    printf("       EXPORT COMMAND\n");
    printf("=============================================\n");

    printf("\nExamples:\n");
    printf("export NAME=Shanmukha\n");
```

```
    char input[MAX_INPUT];

    printf("============================================\n");
    printf("       EXPORT COMMAND\n");
    printf("============================================\n");

    printf("\nExamples:\n");
    printf("export NAME=Shanmukha\n");
    printf("export COURSE=BTech\n");
    printf("export YEAR=2\n");
    printf("child NAME\n");
    printf("exit\n");

    while (1)
    {
        printf("\nshell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            break;
        }

        if (strncmp(input,
                    "export ",
                    7) == 0)
        {
            export_variable(input);
        }
        else if (strncmp(input,
                         "child ",
                         6) == 0)
        {
            char *variable =
                input + 6;
```

```
                    7) == 0)
        {
            export_variable(input);
        }
        else if (strncmp(input,
                         "child ",
                         6) == 0)
        {
            char *variable =
                input + 6;

            if (strlen(variable) == 0)
            {
                printf("Usage: child VARIABLE\n");
            }
            else
            {
                test_child_process(variable);
            }
        }
        else if (strcmp(input, "env") == 0)
        {
            extern char **environ;

            char **env = environ;

            while (*env != NULL)
            {
                printf("%s\n", *env);
                env++;
            }
        }
        else if (strlen(input) == 0)
        {
            continue;
        }
        else
        {
            printf("Invalid command: %s\n",
                   input);
        }
    }

    return 0;
}
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```